

1. Suponiendo una representación de secuencias con árboles binarios, definidos con el siguiente tipo de datos:

`data Tree a = E | N Int (Tree a) (Tree a) | L a`

donde el recorrido inorder del árbol da el orden de los elementos de la secuencia y se almacena la información sobre el tamaño de los árboles en los nodos. Definir las siguientes funciones de manera eficiente:

- a) `divide :: Tree a → Int → (Tree a, Tree a)`, que dada una secuencia s y un natural n , divida la secuencia en dos con los primeros n elementos de s y el resto de los elementos. Por ejemplo,

$$\text{divide} \langle x_0, x_1, x_2, x_3, x_4 \rangle \text{ sp } 3 = (\langle x_0, x_1, x_2 \rangle, \langle x_3, x_4 \rangle)$$

- b) `interleave :: Tree a → Tree a → Tree a`, que dadas dos secuencias s y s' devuelva una secuencia de largo $|s| + |s'|$ con los elementos de s y s' de manera intercalada. Por ejemplo,

$$\text{interleave} \langle x_0, x_1, x_2, x_3 \rangle \langle y_0, y_1, y_2 \rangle = \langle x_0, y_0, x_1, y_1, x_2, y_2, x_3 \rangle$$

Definir `interleave` de manera que en el caso en que los árboles tengan la misma estructura, la función tenga profundidad en $O(h)$, siendo h la altura de los árboles. Dar las recurrencias correspondientes al trabajo y la profundidad de `interleave` para este caso. No hace falta resolverlas.

2. Dada una secuencia de valores numéricos, decimos que el valor de la posición i -ésima es mayor a su promedio histórico si es mayor al promedio de todos los anteriores. Por ejemplo, en la secuencia $\langle 1, 5, 2 \rangle$, los primeros dos elementos son mayores a su promedio histórico, mientras que el último no (para el primer elemento, consideramos que su promedio histórico es 0).

Usando las funciones del TAD secuencia, definir una función `highestIndexes :: Seq Float → Seq Nat` que dada una secuencia s , devuelva una secuencia con los índices de s que corresponden a valores mayores a su promedio histórico. Por ejemplo:

$$\begin{aligned} \text{highestIndexes} \langle 1, 5, 2 \rangle &= \langle 0, 1 \rangle \\ \text{highestIndexes} \langle -2, -5 \rangle &= \langle \rangle \end{aligned}$$

Definir `highestIndexes` con profundidad en $O(\lg n)$, donde n es la longitud de la secuencia.

3. Dada la siguiente definición para representar árboles generales en Haskell:

`data GTree a = Node a [GTree a]`

definir las siguientes funciones:

- a) `elemT :: Eq a ⇒ a → GTree a → Bool`, que determina si un elemento está en el árbol.
- b) `descendent :: Eq a ⇒ a → a → GTree a → Bool`, que dados dos valores x e y y un árbol t , determina si existe un nodo en t cuyo valor sea y que sea descendiente de otro nodo en t cuyo valor sea x . Por ejemplo:

$$\begin{aligned} \text{descendent } 3 \ 5 \ (\text{Node } 3 \ [\text{Node } 4 \ [], \text{Node } 5 \ []]) &= \text{True} \\ \text{descendent } 3 \ 5 \ (\text{Node } 4 \ [\text{Node } 3 \ [], \text{Node } 5 \ []]) &= \text{False} \end{aligned}$$